

UNIVERSIDADE DA BEIRA INTERIOR

DEPARTAMENTO DE INFORMÁTICA



Departamento de
Informática

Aplicação Web

Elaborado por:

Alexandre Santos (52011)

Daniel Carlos (52578)

Vasco Colaço (52290)

Unidade Curricular:

Sistemas Distribuídos

23 de maio de 2026

Conteúdo

1	Introdução	5
1.1	Enquadramento	5
1.2	Objetivos	5
1.3	Organização do Documento	5
2	Tecnologias e Ferramentas Utilizadas	7
2.1	Spring Framework e Spring Security	7
2.2	JSON Web Tokens (JWT)	7
2.3	PostgreSQL	7
2.4	React e Vite	7
3	Modelo de Dados	8
3.1	Descrição das Entidades	8
3.2	Diagrama Entidade-Relação (ERD)	9
3.3	Esquema Relacional e Mapeamentos JPA	10
4	Arquitetura e Implementação do Sistema	12
4.1	Arquitetura de Segurança	12
4.1.1	Estrutura de Utilizadores e Controlo de Perfis	12
4.1.2	Registo de Clientes e Fluxo de Login/Logout	12
4.1.3	Gestão de Sessões Stateless com JWT e Expiração	13
4.1.4	Filtro de Segurança e Middleware de Autenticação	13
4.1.5	Proteção de Rotas e Controlo de Acesso End-to-End	13
4.1.6	Validação de Dados e Proteção de Inputs	14
4.2	Integração Frontend-Backend	14
4.3	Gestão de Catálogo e Vendas	14
4.3.1	Gestão de Catálogo	14
4.3.2	Gestão de Vendas (Carrinho e Checkout)	15
4.4	Lógica de Negócio e Serviços	15
4.5	Processamento de Vendas e Stocks	16
5	Testes e Validação do Sistema	18
5.1	Segurança e Autenticação	18
5.2	Gestão de Produtos	18
6	Conclusões e Trabalho Futuro	20
6.1	Conclusão	20
6.2	Trabalho Futuro	20

Lista de Figuras

3.1	Diagrama Entidade-Relação (ERD) das Tabelas do Sistema	10
-----	--	----

Acrónimos

ACID Atomicity, Consistency, Isolation, Durability (Atomicidade, Consistência, Isolamento e Durabilidade)

API Application Programming Interface (Interface de Programação de Aplicações)

BD Base de Dados

CRUD Create, Read, Update, Delete (Criação, Consulta, Atualização e Eliminação)

HTTP Hypertext Transfer Protocol (Protocolo de Transferência de Hipertexto)

JPA Java Persistence API (API de Persistência Java)

JWT JSON Web Tokens

ORM Object-Relational Mapping (Mapeamento Objeto-Relacional)

REST Representational State Transfer

SGBD Sistema de Gestão de Base de Dados

SQL Structured Query Language (Linguagem de Consulta Estruturada)

URL Uniform Resource Locator (Localizador de Recursos Uniforme)

Glossário

Middleware Componente lógico intermédio (como filtros de segurança) que interceta pedidos HTTP para realizar validações antes de chegarem aos controladores.

Pessimistic Locking Estratégia de controlo de concorrência que bloqueia registos na base de dados durante uma transação para evitar condições de corrida.

Stateless Estilo de arquitetura onde o servidor não mantém qualquer estado de sessão do utilizador, sendo cada pedido autossuficiente para validação.

Thread Linha de execução independente dentro de um processo, permitindo realizar tarefas de forma concorrente no backend.

Capítulo 1

Introdução

1.1 Enquadramento

No âmbito da unidade curricular de Sistemas Distribuídos, foi proposto o desenvolvimento de uma aplicação web completa para uma loja online. O tema escolhido pelo grupo foi uma **Loja de Eletrodomésticos**, implementada recorrendo à framework Spring para o backend e React para o frontend.

Este projeto foca-se na criação de serviços RESTful, persistência de dados e, crucialmente, na segurança e gestão de sessões de utilizadores num ambiente distribuído. Todo o código-fonte pode ser consultado no repositório GitHub do projeto através do endereço https://github.com/Vascorc/Loja_Eletrdomesticos.

1.2 Objetivos

O objetivo principal é desenvolver uma plataforma que permita a gestão de um catálogo de produtos, a realização de compras por clientes registados e a administração de stocks.

Os objetivos específicos incluem:

1. Implementação de uma API RESTful utilizando Spring Boot.
2. Persistência de dados num SGBD relacional (PostgreSQL).
3. Implementação de um sistema de segurança robusto com autenticação JWT (JSON Web Tokens).
4. Gestão de sessões stateless para garantir escalabilidade.
5. Integração do Frontend com o Backend através de serviços assíncronos.
6. Desenvolvimento de lógica distribuída para processamento de vendas, faturas e controlo de stocks concorrente.

1.3 Organização do Documento

Para descrever de forma clara os procedimentos, este documento está organizado nas seguintes secções:

- **Capítulo 1 – Introdução** – apresenta de forma geral o tema e os objetivos do projeto.
- **Capítulo 2 – Tecnologias e Ferramentas Utilizadas** – descreve a stack tecnológica adotada para o desenvolvimento do sistema.

- **Capítulo 3 – Modelo de Dados** – detalha o desenho da base de dados relacional (PostgreSQL), diagramas entidade-relação, as entidades JPA e as políticas de propagação em cascata.
- **Capítulo 4 – Arquitetura e Implementação do Sistema** – explica em pormenor as decisões de arquitetura, o modelo de segurança com JWT, a integração frontend-backend e a lógica de concorrência e reservas de stock.
- **Capítulo 5 – Testes e Validação do Sistema** – detalha os testes práticos efetuados para validar o correto funcionamento de cada funcionalidade e as verificações de segurança.
- **Capítulo 6 – Conclusões e Trabalho Futuro** – sintetiza o trabalho executado e delinea possíveis melhorias futuras.

Capítulo 2

Tecnologias e Ferramentas Utilizadas

2.1 Spring Framework e Spring Security

O backend foi desenvolvido com Spring Boot, utilizando o ecossistema Spring Security para gerir a autenticação e autorização. Esta escolha permitiu uma integração simplificada de mecanismos de segurança avançados.

2.2 JSON Web Tokens (JWT)

Para a gestão de sessões, optou-se por JWT. Esta tecnologia permite que o servidor não necessite de guardar estado (stateless), o que é ideal para sistemas distribuídos, pois o token enviado pelo cliente contém toda a informação necessária para a sua identificação, permitindo que a API escale sem necessidade de partilhar sessões entre instâncias.

2.3 PostgreSQL

A persistência de dados é assegurada pelo PostgreSQL, um SGBD relacional robusto que garante a integridade dos dados de utilizadores, produtos e vendas.

2.4 React e Vite

O frontend foi construído utilizando React, uma biblioteca JavaScript para a criação de interfaces de utilizador dinâmicas.

Capítulo 3

Modelo de Dados

O modelo de dados da aplicação foi projetado para suportar todas as operações de e-commerce, o controlo transaccional de inventário e o sistema de segurança stateless. A persistência é gerida através da especificação JPA (Java Persistence API) com o Hibernate como fornecedor ORM subjacente, mapeando classes Java diretamente para tabelas na base de dados relacional PostgreSQL.

3.1 Descrição das Entidades

O modelo é constituído por 6 entidades principais, cujos propósitos e estruturas são descritos detalhadamente a seguir:

1. **Utilizador (utilizador)**: Armazena os dados cadastrais e as credenciais de segurança de clientes e administradores.
 - **id** (SERIAL PRIMARY KEY): Identificador único numérico gerado automaticamente.
 - **nome** (VARCHAR): Nome completo do utilizador.
 - **email** (VARCHAR, UNIQUE): Endereço de correio eletrónico, usado como credencial única de login.
 - **password** (VARCHAR): Hash de palavra-passe encriptada usando BCrypt.
 - **perfil** (VARCHAR): Nível de privilégio do utilizador (CLIENTE ou ADMIN).
2. **Categoria (categoria)**: Modela as categorias ou famílias de eletrodomésticos (e.g., Frigoríficos, Fogões).
 - **id** (SERIAL PRIMARY KEY): Identificador único da categoria.
 - **nome** (VARCHAR, UNIQUE): Designação única da categoria.
 - **descricao** (TEXT): Detalhes adicionais sobre a categoria.
3. **Produto (produto)**: Contém as especificações técnicas, preço, imagem e estado de stock de cada eletrodoméstico à venda.
 - **id** (SERIAL PRIMARY KEY): Identificador único numérico do artigo.
 - **nome** (VARCHAR): Nome comercial do artigo.
 - **marca** (VARCHAR), **modelo** (VARCHAR): Marca e modelo do fabricante.
 - **preco** (DECIMAL): Preço unitário em Euros.
 - **stock** (INTEGER): Quantidade física disponível em armazém.

- `eficiencia_energetica` (VARCHAR): Classificação de consumo energético (e.g., A, B).
 - `imagem_url` (VARCHAR): Link ou URL da imagem do produto.
 - `categoria_id` (INTEGER REFERENCES `categoria`): Chave estrangeira que classifica o produto.
4. **Venda** (`venda`): Regista a transação global efetuada por um cliente, funcionando como cabeçalho de faturação.
- `id` (SERIAL PRIMARY KEY): Número sequencial da venda/fatura única.
 - `data_venda` (TIMESTAMP): Data e hora exatas da conclusão da compra.
 - `valor_total` (DECIMAL): Custo final acumulado dos produtos adquiridos.
 - `utilizador_id` (INTEGER REFERENCES `utilizador`): Chave estrangeira que associa a compra a um cliente registado.
 - `metodo_pagamento` (VARCHAR): Método escolhido (ex: PayPal, Cartão).
 - `morada`, `cidade`, `codigo_postal`, `telefone` (VARCHAR): Dados de faturação e entrega fornecidos no checkout.
5. **Item de Venda** (`item_venda`): Armazena os detalhes dos produtos contidos em cada venda.
- `id` (SERIAL PRIMARY KEY): Identificador único da linha de faturação.
 - `venda_id` (INTEGER REFERENCES `venda`): Associação à fatura global.
 - `produto_id` (INTEGER REFERENCES `produto`): Produto adquirido.
 - `quantidade` (INTEGER): Unidades compradas.
 - `preco_unitario` (DECIMAL): Preço unitário praticado na transação (estático, salvaguardando alterações de preço posteriores).
6. **Reserva** (`reservas`): Mecanismo de concorrência distribuída que impede o over-selling na base de dados distribuída, cativando temporariamente os artigos por 5 minutos no carrinho do utilizador.
- `id` (SERIAL PRIMARY KEY): Identificador único da reserva.
 - `produto_id` (INTEGER REFERENCES `produto`): Produto alocado.
 - `utilizador_id` (INTEGER REFERENCES `utilizador`): Utilizador que efetuou a reserva.
 - `quantidade` (INTEGER): Unidades reservadas.
 - `data_expiracao` (TIMESTAMP): Data limite para fecho da compra antes de o stock retornar ao catálogo geral.

3.2 Diagrama Entidade-Relação (ERD)

A Figura 3.1 descreve graficamente a estrutura das tabelas da base de dados PostgreSQL e os seus relacionamentos lógicos (chaves primárias, chaves estrangeiras e integridade referencial).

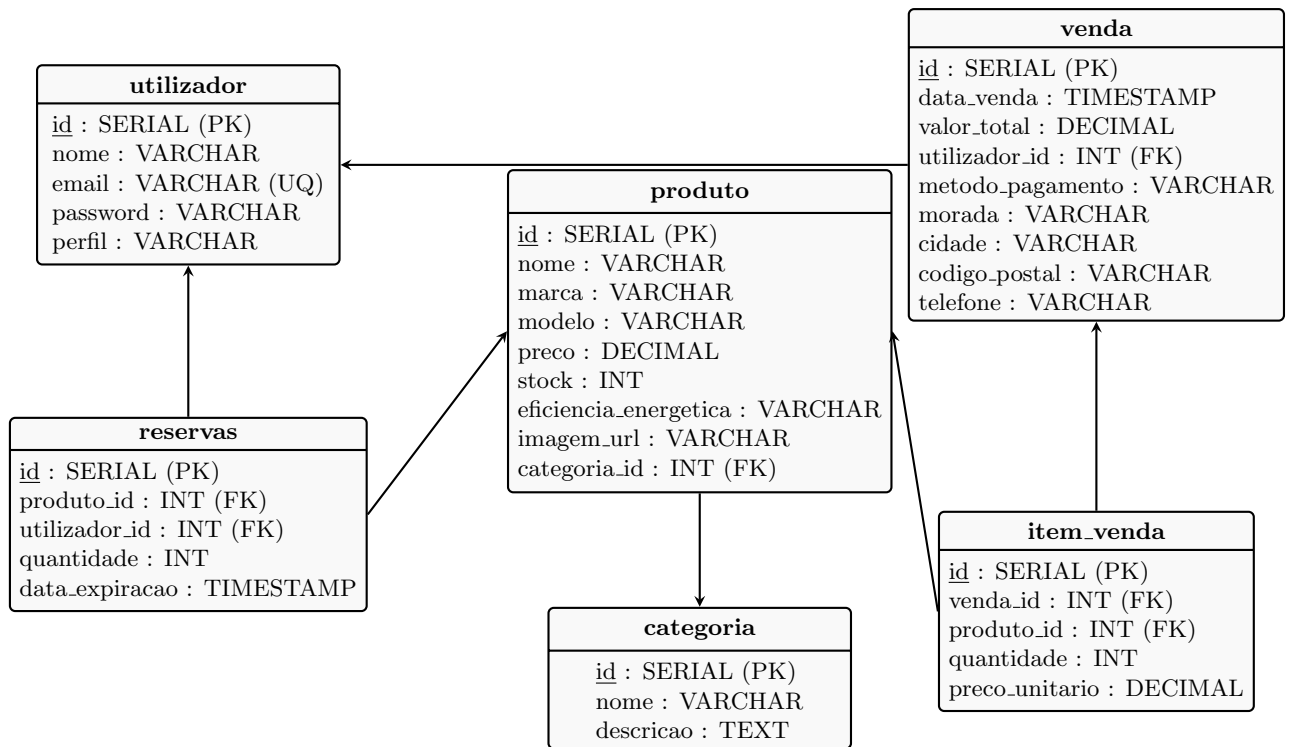


Figura 3.1: Diagrama Entidade-Relação (ERD) das Tabelas do Sistema

Classe Origem	Atributo	Relação JPA	Propagação / Comportamento
Utilizador	vendas	@OneToMany	Mapeado por <code>utilizador</code> , com eliminação em cascata (<code>CascadeType.ALL</code>).
Produto	categoria	@ManyToOne	Chave estrangeira <code>categoria_id</code> . Permite associar a uma categoria mãe.
Venda	itens	@OneToMany	Mapeado por <code>venda</code> , com persistência e remoção integral em cascata.
ItemVenda	venda	@ManyToOne	Ligação direta à venda mãe (<code>venda_id</code>).
ItemVenda	produto	@ManyToOne	Ligação ao registo do produto (<code>produto_id</code>).
Reserva	produto	@ManyToOne	Relação com o produto reservado.
Reserva	utilizador	@ManyToOne	Relação com o utilizador detentor da reserva.

Tabela 3.1: Mapeamentos Objeto-Relacional (JPA) e Comportamentos em Cascata

3.3 Esquema Relacional e Mapeamentos JPA

As relações entre as entidades são mapeadas através de anotações JPA como `@ManyToOne`, `@OneToMany` e `@OneToOne`, estruturando os dados segundo as regras do PostgreSQL. A Tabela 3.1 resume o mapeamento das relações:

Este desenho assegura que, no momento de submissão de uma encomenda, a venda e os seus respetivos itens de venda sejam guardados atomicamente numa única transação controlada pelo Spring Boot. Ao mesmo mais elevado nível, o uso de reservas temporárias previne a ocorrência de condições de corrida num cenário concorrente de compras, mantendo a inte-

gridade referencial e física de todo o stock do inventário.

Capítulo 4

Arquitetura e Implementação do Sistema

4.1 Arquitetura de Segurança

A segurança da aplicação foi desenhada de raiz para proteger de forma robusta e consistente os dados dos utilizadores, bem como controlar o acesso a recursos sensíveis. A implementação garante a integridade de todas as rotas do frontend e endpoints REST do backend, organizando as tarefas de segurança nos seguintes eixos fundamentais:

4.1.1 Estrutura de Utilizadores e Controlo de Perfis

O modelo de utilizadores baseia-se na classe `Utilizador`, persistida na tabela homónima da base de dados. Cada utilizador é associado a um determinado perfil de acesso através do campo `perfil`, cujos valores possíveis são:

- **CLIENTE**: Utilizador comum da loja, com permissão para navegar pelo catálogo, reservar produtos temporariamente no carrinho e concluir compras.
- **ADMIN**: Utilizador com privilégios administrativos completos sobre a gestão de produtos, stock de inventário, categorias, e visualização de faturas e histórico global de vendas.

Na integração com o Spring Security, o perfil é mapeado para um conjunto de autoridades concedidas (`GrantedAuthority`), prefixado com `ROLE_` (e.g., `ROLE_CLIENTE` e `ROLE_ADMIN`), viabilizando o controlo de acesso baseado em perfis tanto a nível global como a nível de controladores específicos.

4.1.2 Registo de Clientes e Fluxo de Login/Logout

- **Registo de Clientes**: Disponibilizado através do endpoint público `/api/auth/signup`. A password fornecida pelo utilizador é encriptada utilizando o algoritmo unidirecional robusto **BCrypt** antes de ser persistida. Isto garante que as passwords nunca são guardadas em texto limpo.
- **Login**: Efetuado via `/api/auth/signin`. O `AuthenticationManager` valida o email e a password introduzidos. Caso as credenciais coincidam com os registos cifrados da BD, o servidor emite um token de autenticação assinado e configurado.
- **Logout**: Gerido de forma *stateless* no frontend. Ao acionar o botão de logout, a aplicação cliente elimina o token JWT armazenado no `localStorage` do browser e reencaminha o utilizador para a página de login, limpando qualquer estado em cache.

4.1.3 Gestão de Sessões Stateless com JWT e Expiração

Para permitir a escalabilidade distribuída do sistema, optou-se por uma gestão de sessões *stateless*. O servidor não armazena nenhuma sessão em memória; toda a informação necessária para identificar e conceder permissões ao utilizador é encapsulada num token seguro do tipo **JSON Web Token (JWT)**. Cada token possui um tempo de expiração estrito (configurado por omissão para 24 horas). Uma vez expirado o token, a sua assinatura deixa de ser válida e o utilizador é forçado a reautenticar-se.

4.1.4 Filtro de Segurança e Middleware de Autenticação

Foi desenvolvido um middleware específico designado `AuthTokenFilter` que estende a classe `OncePerRequestFilter` do Spring Security. Este filtro é executado a cada pedido HTTP enviado para a API:

1. Extrai o cabeçalho *Authorization* do pedido e analisa o prefixo *Bearer*.
2. Utiliza a classe utilitária `JwtUtils` para validar a assinatura, integridade e expiração do token.
3. Recupera a identidade do utilizador (email) a partir do token.
4. Carrega os dados correspondentes através de `UserDetailsServiceImpl` e estabelece o objeto `UsernamePasswordAuthenticationToken` no contexto global de segurança do Spring (`SecurityContextHolder`).

4.1.5 Proteção de Rotas e Controlo de Acesso End-to-End

Para assegurar a máxima proteção, as verificações foram duplicadas em ambas as camadas da aplicação:

- **Frontend (Proteção de Rotas em React):** Criou-se o componente de ordem superior `ProtectedRoute`. Este componente encapsula rotas privadas na aplicação. Se o utilizador não possuir um token ativo no `localStorage`, é reencaminhado para `/login`. Para as rotas dedicadas à administração (como `/admin/produtos` e `/admin/vendas`), o componente valida se o perfil do utilizador é estritamente `ADMIN`. Utilizadores sem o perfil devido são bloqueados e redirecionados para a raiz `/`.
- **Backend (Segurança Geral e Endpoints da API):** A configuração da cadeia de filtros em `WebSecurityConfig` desativa a proteção CSRF (visto que as comunicações usam tokens em vez de cookies) e autoriza de forma pública apenas os endpoints estritamente necessários (registo, login e operações de consulta). Adicionalmente, implementou-se segurança ao nível do método recorrendo a anotações `@PreAuthorize`:

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http.cors(Customizer.withDefaults())
        .csrf(csrf -> csrf.disable())
        .sessionManagement(session -> session.sessionCreationPolicy(
            SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(auth -> auth
            .requestMatchers(HttpMethod.OPTIONS, "**").permitAll()
            .requestMatchers("/api/auth/**").permitAll()
            .requestMatchers("/api/test/**").permitAll()
            .requestMatchers("/api/produtos/public/**").permitAll()
            .requestMatchers(HttpMethod.GET, "/api/categorias/**").permitAll()
            ()
        )
}
```

```

        .anyRequest().authenticated()
    );
    http.authenticationProvider(authenticationProvider());
    http.addFilterBefore(authenticationJwtTokenFilter(),
        UsernamePasswordAuthenticationFilter.class);
    return http.build();
}

```

Listing 4.1: Configuração do SecurityFilterChain no Backend (WebSecurityConfig)

Para garantir que apenas administradores possam gerir produtos e categorias, aplicámos a restrição baseada em perfis diretamente nos controladores do backend:

```

@PostMapping
@PreAuthorize("hasRole('ADMIN')")
public ResponseEntity<ProdutoDTO> criar(@RequestBody ProdutoDTO dto) { ... }

@PutMapping("/{id}")
@PreAuthorize("hasRole('ADMIN')")
public ResponseEntity<ProdutoDTO> atualizar(@PathVariable Long id,
    @RequestBody ProdutoDTO dto) { ... }

```

Listing 4.2: Anotação PreAuthorize no ProdutoController

4.1.6 Validação de Dados e Proteção de Inputs

Para robustecer a segurança contra dados inválidos e vulnerabilidades comuns de segurança (como SQL Injections):

- O backend utiliza mapeamento JPA e repositórios Spring Data com consultas parametrizadas, neutralizando de forma inerente ataques de injeção de código SQL.
- A nível de serviços e controladores, todas as operações de escrita validam o estado das variáveis de entrada, garantindo a integridade dos dados na base de dados relacional.

4.2 Integração Frontend-Backend

A comunicação entre as duas camadas é feita através de serviços assíncronos no frontend. Criou-se um `AuthService` em JavaScript que utiliza a API *fetch* para consumir os endpoints REST do backend. O token recebido é armazenado no `localStorage` para persistência da sessão no browser.

4.3 Gestão de Catálogo e Vendas

A gestão do catálogo e o processamento de vendas formam o verdadeiro coração da aplicação. Na prática, o sistema divide-se em duas grandes dinâmicas que trabalham em conjunto: a manutenção contínua dos produtos e todo o ciclo de vida que acompanha uma venda.

4.3.1 Gestão de Catálogo

Para dar vida ao catálogo, assentamos a lógica nos serviços `ProdutoService` e `CategoriaService`, que garantem o ciclo completo de operações CRUD. O utilizador pode explorar os produtos de forma simples, seja navegando pela listagem global, aplicando filtros por categorias ou fazendo pesquisas diretas pelo nome.

Há também uma funcionalidade inteligente de destaques: o sistema cruza os dados atuais com o histórico de vendas para calcular e expor automaticamente os 5 produtos mais populares do momento.

Para proteger a arquitetura do sistema, a comunicação entre o *backend* e o *frontend* nunca expõe o modelo de domínio interno de forma direta. Em vez disso, utilizamos DTOs (*Data Transfer Objects*), como o `ProdutoDTO`, que servem como uma camada segura de isolamento.

4.3.2 Gestão de Vendas (Carrinho e Checkout)

A experiência de compra começa quando o utilizador adiciona um artigo ao carrinho, disparando de imediato um mecanismo de reserva temporária de stock. Quando decide avançar para o *checkout*, o sistema recolhe os dados de envio e pagamento através do `CheckoutRequestDTO` e submete-os para processamento.

É neste momento que o *backend* assume o controlo através de um fluxo estruturado:

- Percorre detalhadamente cada um dos itens guardados no carrinho;
- Calcula o valor total finalizado do pedido;
- Gera e persiste as respetivas entidades de `Venda` e `ItemVenda`.

Assim que a transação é concluída com sucesso, a venda fica imediatamente vinculada ao perfil do `Utilizador` autenticado. O registo passa a estar disponível para consulta no seu histórico pessoal de compras e na área de faturação.

4.4 Lógica de Negócio e Serviços

A arquitetura do backend foi desenhado seguindo um modelo de camadas focado no domínio. Toda a lógica de negócio fica estritamente isolada na camada de serviços — identificados pela anotação `@Service` do Spring Boot. Esta abordagem garante uma separação clara de responsabilidades, evitando que as regras de processamento se misturem com os controladores REST ou com a persistência de dados nos repositórios.

A dinâmica de compras e vendas assenta em três pilares principais:

ProdutoService

Centraliza toda a gestão e validação dos produtos. É responsável por verificar a existência de categorias associadas durante os processos de criação ou atualização, garantindo também uma conversão precisa e segura entre as entidades da base de dados e os DTOs.

VendaService

Orquestra o fluxo crítico de finalização de compra. O método de *checkout* opera sob um contexto transacional (`@Transactional`), validando a identidade do utilizador antes de construir a entidade de venda. A partir daí, percorre os itens do pedido para delegar a verificação de disponibilidade física e a respetiva retenção de stock. Adicionalmente, este componente gere o histórico de compras dos clientes — aplicando os filtros de segurança necessários — e disponibiliza uma visão consolidada das vendas para o perfil de administrador.

ReservaService

Controla o ciclo de vida temporal do carrinho de compras. Assim que um produto é reservado, o sistema deduz temporariamente essa quantidade do stock por um período de 5 minutos. Para melhorar a experiência do utilizador e evitar frustrações, o sistema renova

automaticamente o prazo de expiração de todas as reservas ativas daquele perfil sempre que um novo item é adicionado ao carrinho.

4.5 Processamento de Vendas e Stocks

Para evitar que múltiplos utilizadores comprem o mesmo produto em simultâneo, o sistema utiliza uma estratégia dupla de sincronização: bloqueios imediatos na base de dados e um circuito de reservas temporárias.

Isolamento de Concorrência e Abordagem Comparativa (JVM vs. Base de Dados): Para justificar o desenho do sistema no âmbito de Sistemas Distribuídos, analisou-se detalhadamente a estratégia de exclusão mútua no acesso concorrente ao stock, avaliando duas hipóteses principais:

1. **Sincronização em Memória (synchronized na JVM):** A utilização de monitores e blocos declarados como `synchronized` em Java é uma solução clássica para exclusão mútua ao nível de *threads* a correr localmente numa única máquina virtual. No entanto, esta hipótese apresenta duas limitações graves para o projeto:

- *Limitação de Distribuição:* Num sistema distribuído, onde o backend pode ser escalado horizontalmente (múltiplas instâncias físicas da API a correr em paralelo atrás de um *Load Balancer*), cada JVM terá a sua própria memória *heap* isolada. Como os trincos locais não se propagam pela rede, dois servidores distintos permitiriam vendas concorrentes incorretas, gerando rutura e inconsistência de stock (*over-selling*).
- *Desconexão com a Base de Dados:* Uma vez que o stock não reside no Heap do Java mas sim no PostgreSQL, sincronizar apenas a execução na JVM não impede que duas transações assíncronas concorrentes leiam dados obsoletos da base de dados se as transações não forem bloqueadas a nível relacional.

2. **Bloqueio Pessimista na Base de Dados (Abordagem Adotada):** O sistema protege o acesso ao inventário através de bloqueio pessimista (*Pessimistic Locking*), recorrendo à anotação `@Lock(LockModeType.PESSIMISTIC_WRITE)` no método `findByIdWithPessimisticL` do repositório. Esta instrução gera a cláusula SQL nativa `SELECT ... FOR UPDATE` na base de dados, sendo a solução ideal devido aos seguintes fatores:

- *Sincronização Distribuída Inerente:* Como todas as réplicas ou instâncias da API de backend partilham a mesma base de dados PostgreSQL, o SGBD funciona como a única fonte de verdade e o coordenador distribuído de concorrência. O bloqueio é aplicado à linha física da tabela `produto`, forçando qualquer outra instância a aguardar em fila.
- *Consistência Forte:* Elimina o risco de anomalias de escrita concorrente (*lost updates*), pois a linha fica retida para escrita e leitura protegida até que a transação inicial efetue o respetivo *commit* ou *rollback*, mantendo a integridade absoluta do stock de inventário.

Fluxo de Reserva Dinâmica: Para mitigar a frustração de ruturas de stock a meio do pagamento, o processo foi estruturado em três etapas:

- **Adição ao carrinho:** Assim que o item entra no carrinho, o stock é deduzido e alocado provisoriamente na entidade `Reserva`.

- **Finalização da compra:** No momento do checkout, o `VendaService` liquida a reserva ativa sem reprocessar o stock do produto.
- **Verificação de segurança:** Falhas ou omissões na reserva forçam a validação direta no inventário livre, disparando uma *Runtime Exception* caso o saldo seja insuficiente.

Libertação Automática de Recursos: Para que o stock não fique retido indefinidamente em carrinhos abandonados, o componente `ReservaCleanupTask` executa uma rotina cíclica em background. Esta tarefa identifica reservas expiradas, devolve os artigos ao inventário disponível do `Produto` e elimina o registo temporário, assegurando a rotação contínua do catálogo.

Capítulo 5

Testes e Validação do Sistema

Neste capítulo, é apresentada a validação prática e o plano de testes executado para comprovar a correta implementação e a robustez dos requisitos funcionais e técnicos definidos para o sistema. O processo de verificação cobriu desde o comportamento da interface de utilizador (frontend) até às regras de negócio, controlo transaccional concorrente e políticas de segurança impostas pelo Spring Security (backend). As secções seguintes detalham os cenários simulados, a recolha de evidências de funcionamento e a resposta da plataforma face a acessos válidos e tentativas de intrusão.

5.1 Segurança e Autenticação

Para testar a fiabilidade do sistema de segurança concebido, foram conduzidos testes de validação focados em cenários de acesso não autorizado:

1. **Verificação de Sessão Ativa:** Ao aceder ao dashboard ou ao catálogo de produtos sem um token de autenticação válido armazenado, o componente `ProtectedRoute` do React bloqueia imediatamente o carregamento do conteúdo e redireciona o utilizador para a página de `/login`.
2. **Fluxo de Registo e Login com Sucesso:** Testou-se a criação de um utilizador comum e a autenticação do mesmo. O token JWT foi emitido com sucesso pelo servidor e carregado no cabeçalho HTTP de todos os pedidos assíncronos seguintes.
3. **Validação de Acesso Restrito no Frontend:** Efetuou-se login com um utilizador comum (perfil `CLIENTE`). Ao tentar forçar o URL de administração no browser (`http://localhost:5173/admin/produtos`), o componente de segurança barrou o acesso e reencaiminhou o utilizador de volta para o dashboard principal.
4. **Teste de Bloqueio de Endpoints no Backend:** Através da simulação direta de pedidos REST HTTP para os endpoints reservados (e.g., pedidos POST para `/api/produtos` ou `/api/categorias`) utilizando ferramentas como Postman e enviando um token de utilizador com perfil `CLIENTE`, a API do Spring Boot respondeu com o estado de erro **403 Forbidden**, validando com absoluto rigor a eficácia das anotações `@PreAuthorize("hasRole('ADMIN')`.

5.2 Gestão de Produtos

Para validar as funcionalidades de administração do catálogo, foi testada a página de **Gestão de Produtos** (acessível exclusivamente a utilizadores com o perfil `ADMIN`). As operações CRUD (Create, Read, Update, Delete) foram avaliadas da seguinte forma:

1. **Leitura (Read):** Verificou-se que a tabela principal carrega com sucesso todos os produtos disponíveis no sistema, exibindo de forma clara a imagem, nome, categoria, preço, stock atual e eficiência energética. Adicionalmente, testou-se o feedback visual para níveis críticos de stock (destaques em estilos condicionados para rutura de stock ou stock baixo).
2. **Criação (Create):** Através da ação “Novo Produto”, foi aberto um formulário modal. Procedeu-se ao preenchimento de todos os dados do novo eletrodoméstico (incluindo o URL da imagem, seleção da categoria e nível de eficiência). Ao submeter, o novo registo foi imediatamente persistido na base de dados e a tabela foi atualizada de forma imediata.
3. **Atualização (Update):** Utilizando a opção “Editar” num produto existente, o formulário modal foi pré-preenchido com os dados atuais do eletrodoméstico. Após a modificação do preço e da quantidade em stock, as alterações foram gravadas com sucesso e validadas na listagem da administração.
4. **Eliminação (Delete):** Foi testada a operação de remoção de um produto clicando no botão “Eliminar”. O sistema apresentou um alerta de confirmação no browser, prevenindo eliminações acidentais. Após a confirmação, o produto foi permanentemente removido e a lista limpa da interface.

Todos os pedidos efetuados durante a validação das funcionalidades incluíram o *JSON Web Token* no formato Bearer associado a um administrador. Isto assegurou que as validações de autorização, asseguradas pelo *WebSecurityConfig* no lado do servidor, fossem efetuadas em harmonia com as proteções no frontend.

Capítulo 6

Conclusões e Trabalho Futuro

6.1 Conclusão

O desenvolvimento deste projeto permitiu consolidar os conhecimentos sobre a criação de aplicações web distribuídas utilizando a framework Spring. A implementação de um sistema de segurança baseado em JWT e a integração com uma base de dados relacional demonstraram a importância da robustez e escalabilidade em sistemas modernos.

6.2 Trabalho Futuro

Para versões futuras deste sistema, poderiam ser implementadas as seguintes melhorias:

- **Sistema de Pagamento:** Integração com gateways de pagamento reais (ex: Stripe ou PayPal).
- **Notificações:** Implementar um sistema de notificações por email para confirmação de encomendas.

Bibliografia

- [1] Daniel J. Berg and J. Steven Fritzinger. *Advanced Techniques for Java Developers*. John Wiley & Sons, 1999.
- [2] James Gosling, Bill Joy, Guy Steele, and Gilad Bracha. *The Java Language Specification*. Addison-Wesley, 2000.
- [3] Jeff Magee and Jeff Kramer. *Concurrency: State Models and Java Programs*. John Wiley & Sons, 1999.
- [4] Oracle. Java platform, standard edition 17 api specification, 2021.
- [5] Jon Postel. Transmission control protocol, 1981.
- [6] Tiago Miguel Carrola Simões. Notas de aula e materiais de apoio de sistemas distribuídos. Universidade da Beira Interior (UBI), 2026. Acedido via plataforma EduMark e Moodle.